Infinity 1.93 Core – Reading Guide

Oct 28, 2026

Big picture

The core is one 49,964-byte program in three parts: a small start-up stub, a resident block that copies itself to 0x80050000 and stays in RAM, and data tables at the end of that block. The files `core_codedecrypted.bin` and `core_listing.txt` are both this same image, with the break-crypt layer already removed.

Part	Listing addresses (file offset)	Runs at	Size	What it does
Start-up stub	0x110000-0x11064F (0x0000-0x064F)	0x110000	1,616 bytes	Clears memory, then copies the resident block to 0x80050000 and jumps there
Resident code	0x110650-0x1192CF (0x0650-0x92CF)	0x80050000-0x80058C7F	about 35 KB	Helpers, hooks, the dispatcher and every patch routine
Resident data	0x1192D0-0x11BF2B (0x92D0-0xC32B)	0x80058C80-0x8005BCDB	about 12 KB	Signature/mask table, payload blobs, the Y table, program names
Globals (not in the file)	n/a	0x8005BD00-0x8005BD4F, 0x8005CF90	small	Filled at run time: unpacked settings, saved kernel pointers, the settings record copy

The resident part is 48,348 bytes from listing 0x110650. The listing keeps the 0x110000 numbering, but the code actually runs at 0x80050000, so the next section shows how to convert between them.

What's in the resident code

The resident code is the program that applies the patches: helpers first, then the patch routines, the kernel hooks and the start-up routine, in that address order. Three blobs that get copied out whole (two stubs and the game-boot block) sit in the middle of it. Rows marked "not covered" are the parts this guide leaves out.

Resident	Listing	What it is	Menu option
0x80050020	0x110670	Macrovision stub (blob, 28 bytes)	cfg[6]
0x80050940	0x110F90	Display trap stub (blob, 16 bytes, copied to 0xA0000100)	cfg[1]
0x80050950	0x110FA0	Display trap handler the stub jumps to; applies the Y offsets in 0x8005BD20 and 0x8005BD10	cfg[1], cfg[10]
0x80050E00	0x111450	Break handler (decrypts functions as they run)	all
0x80050FB8	0x111608	Signature search	all
0x80051120	0x111770	Program-name compare	all
0x800512C0	0x111910	Word/mask pair search	all
0x80051388	0x1119D8	Memory fill: a2 bytes at a0 set to a1	all
0x800513B8	0x111A08	DVD player patch	cfg[11], arms cfg[7]
0x800514F8	0x111B48	OSDSYS routine	not covered
0x80051978	0x111FC8	PS2LOGO routine (game boot); only its cfg[12] flag is covered	cfg[12] flag only
0x80051C00	0x112250	PS1DRV patch	cfg[2], cfg[14]
0x80051EA8	0x1124F8	Name dispatcher	all
0x80052198	0x1127E8	Chip check, part of the handshake (not covered)	not covered
0x80052318	0x112968	IOP patch, ATAD part 1 (2 signatures)	cfg[4]

Resident	Listing	What it is	Menu option
0x800524A0	0x112AF0	IOP routine: MC16 string search, plus a part not covered	cfg[3]
0x80052650	0x112CA0	IOP patch, ATAD part 2 (3 signatures)	cfg[4]
0x80052B10-0x800530C8	0x113160	Chip command and handshake code	not covered
0x80053118	0x113768	Kernel slot 0x13C wrapper: green fix, late DVD patch	cfg[7], cfg[11]
0x800531D8	0x113828	Kernel slot 8 wrapper: video-mode changes (incl. mode 5 settings)	cfg[1]
0x80053310	0x113960	Start-up routine	cfg[1], [4], [6], [10], [11]
0x80053B50	0x1141A0	Game-boot block (blob, 2,744 bytes), copied out by the PS2LOGO routine; not covered	not covered
0x80054608-0x80058C7F	0x114C58	Remaining code, used by the parts not covered	not covered

Converting addresses

To find a resident address in the listing, subtract 0x80050000 and add 0x110650. Go the other way to turn a listing address into the address the code uses. Every `jal`, pointer and table address inside the code uses the 0x8005xxxx form, so you will convert a lot.

```
\text{listing} = \text{resident} - \mathtt{0x80050000} + \mathtt{0x110650}
```

You see in code	Look up in listing	File offset	What it is
`jal 0x80050fb8`	0x111608	0x1608	Shared signature search
`jal 0x800512c0`	0x111910	0x1910	Word/mask pair search

You see in code	Look up in listing	File offset	What it is
`jal 0x80051120`	0x111770	0x1770	Program-name compare
`jal 0x80051ea8`	0x1124F8	0x24F8	Name dispatcher
`0x80058cb0`	0x119300	0x9300	Start of the signature/mask table
`0x80058bb0`	0x119200	0x9200	Program names: OSD, PS1DRV, DVD, PS2LOGO

Two tricks for reading constants:

- An address is usually built in two steps: `lui t0,0x8006` then `addiu t0,t0,-29080`. The second number is negative, so it is 0x80060000 - 29080 = 0x80058E68.
- Any 0x8006 + negative pair lands in 0x80058xxx-0x8005Fxxx: the data tables or the globals.

Start-up: settings and globals

One routine at resident 0x80053310 (listing 0x113960) prepares everything the menu patches need. It reads the settings record at 0x8005CF90, hooks a few EE kernel table entries (saving the originals in globals), checks the BIOS version, then applies the kernel-side menu patches.

In order, it:

1. Stores the DVD region as cfg[11] + 1 in 0x8005BD38.
2. Hooks kernel table slots 0x13C and 0x50 (originals saved to 0x8005BD08 and 0x8005BD18).
3. If cfg[4] ATAD is on, hooks slot 0x1DC (original saved to 0x8005BD04). Slot 0x1E4 is always hooked (original in 0x8005BD34).
4. Scans the BIOS ROM from 0xBFC02000 for the `PS20xyz` version string and sets the version flags.
5. Runs the cfg[6] Macrovision and cfg[1] screen-fix kernel patches, and sets the cfg[10] Y offsets (all in the walkthrough below).

Global	Set from	Meaning
0x8005BD38	cfg[11] + 1	DVD-Video region used by the DVD player patch
0x8005BD1C	BIOS version	1 when BIOS is 1.70 or later
0x8005BD3C	BIOS version	1 when BIOS is 2.20 or later
0x8005BD2C	BIOS string	Version digits copied from the ROM
0x8005BD28	cfg[1]	1 when cfg[1] = 2 (selects the Y base values)
0x8005BD20, 0x8005BD10	cfg[1], cfg[10]	Y and DY offsets for the display trap
0x8005BD08, 0x8005BD18, 0x8005BD04, 0x8005BD34, 0x8005BD24	kernel table	Original kernel entries, so hooks can call through
0x8005BD30	BIOS version	Pointer to the chip-command routine (two versions)

The same routine also sets up the core's own boot hooks; those are outside this guide.

Shared helpers

These small routines are called from every patch routine, so learning their arguments makes the patch code much easier to read. MIPS passes arguments in a0-a3, then t0-t3, and returns in v0.

Routine	Listing	Arguments	Returns
Signature search 0x80050FB8	0x111608	a0 start, a1 end, a2 step (always 4), a3 length in bytes, t0 pattern address, t1 mask address	v0 = address of the first match, 0 if none

Routine	Listing	Arguments	Returns
Word/mask pair search 0x800512C0	0x111910	a0 list of (word, mask) pairs, a1 number of words, a2 start, a3 end	v0 = address of the match, -1 (0xFFFFFFFF) if none
Program-name compare 0x80051120	0x111770	a0 name string (e.g. "PS1DRV"), a1 the loading program's name	v0 non-zero on a match
Chip check 0x80052198	0x1127E8	none	Called at the end of most patch routines; part of the chip handshake (not covered)
Break handler 0x80050E00	0x111450	none (exception)	Decrypts a `break`-marked function before it runs and re-encrypts it after

The signature search compares `(memory & mask) == (pattern & mask)` four bytes at a time. That is why the verifier's masked patterns line up with the table. Mask address 0x80058CB0 is 16 bytes of 0xFF, meaning "match every byte".

In the decrypted file every function already shows in the clear. Each one still starts with a `break` word (top byte 0x01) and ends with `break` 0x0300000D; treat those as start and end markers.

Menu options, one by one

Ten of the thirteen menu bytes are read by the core; cfg[5], cfg[8] and cfg[13] never are, so the chip itself must handle them. "Read at" is the resident address where the byte is loaded; add 0xC0650 to find it in the listing.

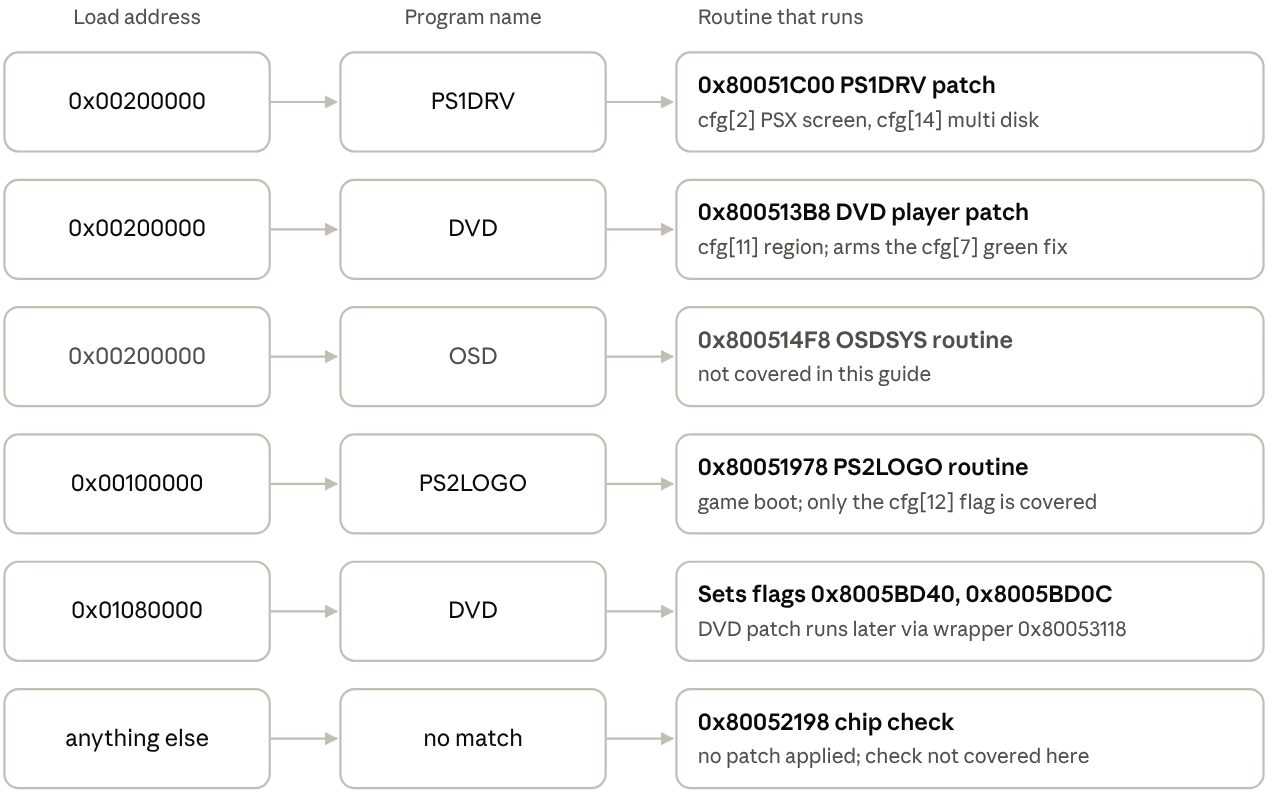
Byte	Option	Read at	Patch function	What gets patched
cfg[1]	PS2 screen fix	0x80053598, 0x80053628, 0x800531F4	start-up 0x80053310	Mode 1 or 4: one byte of the kernel's video-mode `ori` (0x4504) set to 0x45 or 0x65. Mode 2 and up: 16-byte trap stub at 0xA0000100 plus a kernel slot 8 hook to 0x800531D8 for video-mode changes; the trap handler at 0x80050950 applies the Y offsets. Mode 5 also sets 0x80050DF8 = 1
cfg[2]	PSX screen fix	0x80051C28	PS1DRV 0x80051C00	Two PS1DRV signatures, then a scan for `ori v0,v0,0x1401`. PAL console: 144-byte payload; others: 68 + 24-byte payloads. `j` hooks into the copied code
cfg[3]	MC16	0x80052590	IOP 0x800524A0	Finds the string `secrman_for_cex` in IOP RAM
cfg[4]	ATAD auto patch	0x800533CC	IOP 0x80052318, 0x80052650	Hooks kernel slot 0x1DC, then five IOP signatures
cfg[6]	Macrovision fix	0x80053540	start-up 0x80053310	Scans the kernel for `jal X; li a0,0xA7` and hooks the start of function X with a jump to a stub at 0x80050020 that sets a1 = 0 when a0 is 0xA6-0xAF

Byte	Option	Read at	Patch function	What gets patched
cfg[7]	Green fix	0x8005314C	wrapper 0x80053118	On kernel slot 0x13C: clears bit 0 of the first argument, only after the DVD player patch has run (flag 0x80058C8C)
cfg[10]	Y screen fix	0x800536A0, 0x80053750	start-up 0x80053310	Index into the 10-byte table at 0x80058C80, added to Y (x1) and DY (x2)
cfg[11]	DVD-Video region	0x8005335C	DVD 0x800513B8	Region + 1 stored in 0x8005BD38; three DVD-player signatures and one IOP signature
cfg[12]	DVD9 DL support	0x80051BC4	PS2LOGO 0x80051978	Sets one flag at 0x000E000C, read by code the PS2LOGO routine copies in at game boot. The rest of that routine is not covered
cfg[14]	PSX multi disk	0x80051E50	PS1DRV 0x80051C00	No console patch. When off (BIOS 1.70+), the core sends chip command A810 or A814

Which program triggers which patch

The dispatcher at 0x80051EA8 (listing 0x1124F8) runs each time the kernel starts a program. It checks the load address first, then the program name with the name compare, and calls one patch routine.

Each loading program triggers one patch routine



name dispatcher 0x80051EA8 · 6 outcomes

Read each row left to right: a program loading at that address with that name runs the routine on the right. A load at 0x00200000 named DVD runs the DVD patch straight away. A load at 0x01080000 named DVD only sets flags, and the patch runs later through the kernel wrapper.

Where the data lives

Most patch data sits in one run at the start of the resident data, listing 0x119200-0x119688; the game-boot block and two stubs live elsewhere. Patterns and masks are packed back to back with no headers, so the only way to know where one ends is the length (a3) the code passes to the search.

Data	Resident	Listing	Size	Used by
Program names	0x80058BB0	0x119200	4 x 8 bytes	Dispatcher: OSD, PS1DRV, DVD, PS2LOGO

Data	Resident	Listing	Size	Used by
Y offset table	0x80058C80	0x1192D0	10	cfg[10]: 0, 0, +5, -5, +10, -10, +15, -15, +20, -20
All-0xFF mask	0x80058CB0	0x119300	16	Any search that matches every byte
Signatures and masks	0x80058CC0-0x80058F47	0x119310-0x119597	648	Every signature-search call
PSX screen fix, non-PAL A	0x80058F48	0x119598	68	cfg[2]
PSX screen fix, non-PAL B	0x80058F90	0x1195E0	24	cfg[2]
PSX screen fix, PAL	0x80058FA8	0x1195F8	144	cfg[2]
Game-boot block (dvd9_game_payload_2744 in the verifier)	0x80053B50	0x1141A0	2,744	PS2LOGO routine, copied to 0xA00E0000 at every game boot; holds more than DVD9 handling (not covered)
Macrovision stub	0x80050020	0x110670	28	cfg[6]
Display trap stub	0x80050940	0x110F90	16	cfg[1], copied to 0xA0000100

The signature run also holds patterns for routines that aren't menu options (see the last section). The verifier's table covers the menu ones, and the `CONFIG_DATA` checks cover the stubs and constants above.

Reading a patch routine

Every menu patch follows the same four-step shape: set up the search arguments, call the search, check v0, then store new instructions at the hit. The DVD region routine at resident 0x800513B8 (listing 0x111A08) is a good one to practise on; this is its first patch:

```
800513d8  addiu t0,t0,-29080   ; t0 = 0x80058E68  pattern
800513dc  addiu t1,t1,-29064   ; t1 = 0x80058E78  mask
800513e0  lui   a0,0x20        ; a0 = 0x00200000  search start
800513e4  lui   a1,0x30        ; a1 = 0x00300000  search end
800513e8  addiu a2,zero,4      ; step 4
800513ec  jal   0x80050fb8     ; signature search
800513f0  addiu a3,zero,16     ; length 16 (runs in the delay slot)
800513f4  daddu a1,v0,zero     ; a1 = hit address
800513f8  beq   a1,zero,...    ; no hit -> skip
800513fc  lui   v0,0x3e0       ; v0 = 0x03E00000 ...
80051400  lui   v1,0x2402      ; v1 = 0x24020000 = addiu v0,zero,0
80051404  ori   v0,v0,0x8      ; v0 = 0x03E00008 = jr ra
80051408  sw    v0,0(a1)       ; hit + 0: jr ra
8005140c  sw    v1,4(a1)       ; hit + 4: li v0,0 (delay slot)
```

How to read it:

1. **Find the call.** Look for `jal 0x80050fb8` (or `jal 0x800512c0`).
2. **Work out the arguments.** Read the instructions just above it. Remember the delay slot: the instruction right after a `jal` runs before the call, so a3 is set there.
3. **Decode the range.** 0x200000-0x300000 is where the DVD player and PS1DRV load, 0x100000-0x200000 is PS2LOGO, 0xA0000000+ is the EE kernel, and 0xBC000000+ is IOP RAM.
4. **Look up the pattern.** Convert t0 and t1 to listing addresses and read a3 bytes from each.
5. **Read the write.** After the `beq` on v0, `lui` / `ori` pairs build MIPS instruction words, and `sw` stores them at the hit (or at hit + an offset). A copy loop (`lbu` / `sb` with a counter) means a payload blob is being copied in.

This first patch turns the matched function into `jr ra; li v0,0`, so it returns 0 straight away. The next two patches in the same routine (0x8005142C and 0x80051470) load 0x8005BD38 (cfg[11] + 1) and build an `li` with the region number. That is how a menu setting ends up inside the patched code.

Line-by-line listings of the helpers and every menu routine are in Annotated routines.

Not covered here

This guide covers only the menu options. The parts of the core below are left out on purpose; they deal with the console's own checks or the chip's handshake, not user settings.

- The OSDSYS routine at 0x800514F8 (dispatcher entry "OSD" at 0x200000)
- The IOP search at 0x800524D8 inside 0x800524A0 (only the MC16 part of that function is covered)
- Chip command and handshake code from 0x80052B10 to 0x800530C8
- The PS2LOGO routine at 0x80051978 and the block it copies in at game boot (apart from the cfg[12] flag)
- The chip check at 0x80052198, called at the end of most patch routines
- The boot hooks set up at the end of the start-up routine (from 0x800537A4)